

Credit Card Fraud Detection

A Machine Learning Project

From Raw Transactions to a 90%+ Precision Fraud Detector

Metric	Value
Algorithm	XGBoost
Final Precision	90.3%
Final Recall	91.1%
F1 Score	90.7%
ROC-AUC	~0.999
Experiments Run	9
Frauds Caught (test set)	406 out of 429

1. Introduction

Every time you swipe your credit card, the bank checks whether the transaction looks legitimate. This project builds that exact decision-making system using machine learning.

The goal is simple: given information about a transaction — the amount, where it happened, when it happened, and who made it — predict whether it is fraudulent or genuine.

Why is this hard?

Fraud is rare. In a typical dataset, fewer than 1% of transactions are fraudulent. This means if a model just predicts "not fraud" every single time, it would be 99% accurate — yet completely useless. The real challenge is correctly identifying the rare fraudulent cases without raising too many false alarms on legitimate transactions.

This project tackles that challenge through careful feature engineering, systematic experimentation with 9 different techniques, and threshold optimisation to produce a model that a real bank could actually use.

2. The Dataset

The dataset used is the fraudTest.csv file, a publicly available credit card transaction dataset. It contains records of real-world card transactions with a label indicating whether each transaction was fraudulent.

What each record contains

- Transaction timestamp — when the purchase happened
- Transaction amount — how much was spent
- Merchant name and category — where the money was spent
- Customer details — name, address, date of birth, gender, job
- Location coordinates — latitude and longitude of both the customer and the merchant
- is_fraud — the label: 1 if fraudulent, 0 if legitimate

Dataset source

fraudTest.csv — based on the publicly available simulated credit card transaction dataset commonly used for fraud detection research on Kaggle.

3. Data Preparation & Feature Engineering

Raw transaction data cannot be fed directly into a machine learning model. The data needs to be cleaned, transformed, and enriched with additional features that help the model detect fraud patterns. This section walks through every step.

3.1 Removing Irrelevant Columns

Some columns add no value for fraud detection. Personal identifiers like first name, last name, street address, and transaction number are removed. These columns are unique per record, so the model cannot learn any general pattern from them — they just add noise.

3.2 Extracting Time-Based Features

The raw timestamp column is broken into two useful numeric features:

- Hour of the day — fraud often spikes at unusual hours, such as 2 AM or 3 AM
- Day of the week — spending patterns differ between weekdays and weekends

These two numbers are far more useful to a model than a raw timestamp string.

3.3 Calculating Customer Age

The customer's date of birth is converted into their age at the time of the transaction. Age is a useful signal because spending behaviour and fraud vulnerability can vary across age groups. Older customers, for example, may be more susceptible to certain types of fraud.

3.4 Calculating Geographic Distance

A new feature called distance is created by calculating the real-world distance (in kilometres) between the customer's registered address and the merchant's location. The calculation uses the geodesic method, which accounts for the curvature of the Earth.

Why this matters

If a customer's registered address is in Mumbai but a transaction is happening at a merchant 2,000 km away, that is suspicious. Geographic distance is a strong fraud signal for card-present transactions.

3.5 Sorting by Time

Before creating any customer-history features, the entire dataset is sorted chronologically by `unix_time`. This is critical — if we don't sort first, "past" transactions might accidentally include future ones, which would cause data leakage and make the model cheat.

3.6 Transaction Count Per Customer

A running count of how many transactions each customer has made so far is added. This is calculated using `cumcount()` which, because the data is already sorted by time, only counts transactions before the current one.

A customer making their very first transaction looks different from a customer who has made 500 transactions. This count helps the model understand how established a customer's behaviour is.

3.7 Historical Average Spending

For each transaction, we calculate what the customer typically spends — but only using their past transactions, never the current one. The `shift(1)` operation is key here: it shifts the rolling average forward by one row, so the current transaction is always excluded.

Data leakage prevention

If we included the current transaction in the average, the model would be using information it wouldn't have at prediction time. `shift(1)` ensures the feature is realistic: at the moment of a transaction, the model only knows what happened before, not during.

For a customer's very first transaction, there is no history, so the current transaction amount is used as a placeholder.

3.8 Amount Deviation

This feature answers the question: "How unusual is this transaction compared to what this customer normally spends?"

It is calculated by dividing the current transaction amount by the customer's historical average. A ratio of 1.0 means perfectly normal. A ratio of 10.0 means the customer is spending ten times their usual amount — a strong fraud signal.

3.9 Time Since Last Transaction

This feature measures how many seconds have passed since the customer's previous transaction. Fraud often involves rapid, back-to-back transactions as criminals try to drain a card quickly. A very short time between transactions is suspicious.

For a customer's first transaction, this value is set to 0 since there is no previous transaction.

3.10 Encoding Categorical Features

Machine learning models work with numbers, not text. Three strategies were used to convert text columns:

- One-hot encoding — used for category (e.g. grocery, entertainment) and state. Each unique value gets its own binary column.
- Frequency encoding — used for city and job. Each value is replaced with how often it appears in the dataset. This avoids creating hundreds of new columns while still capturing how common or rare each city and profession is.

- Binary encoding — used for gender. Male is encoded as 1, Female as 0.

3.11 Merchant Transaction Count

Similar to the customer transaction count, this feature tracks how many times a particular merchant has appeared in the dataset before the current transaction. A brand-new merchant with no transaction history may be riskier than an established one.

3.12 Final Cleanup

After all features are created, columns that were only used as intermediate steps are dropped. These include the original timestamp, date of birth, latitude and longitude coordinates, credit card number, and merchant name. The model no longer needs them — all the useful information has been extracted into the engineered features.

4. Experiments

Nine experiments were run, each testing a different technique for handling the class imbalance problem. Every experiment uses the same train-test split (80% training, 20% testing) with stratification to ensure fraud cases are proportionally represented in both sets.

Experiment 1 — Baseline Random Forest

A standard Random Forest classifier is trained with no special handling for class imbalance. This gives us a reference point. Random Forest works by training many decision trees and letting them vote on the final prediction.

What we expect

High precision (when it says fraud, it's usually right) but low recall (it misses many actual fraud cases) because it is biased towards the majority class.

Experiment 2 — Class Weighting (balanced)

The `class_weight='balanced'` parameter tells the model to automatically assign more importance to the minority class (fraud). If fraud makes up 0.5% of the data, fraudulent transactions are given 200x more weight in the loss function. This forces the model to pay more attention to fraud cases during training.

Experiment 3 — SMOTE

SMOTE stands for Synthetic Minority Oversampling Technique. Instead of just weighting existing fraud cases more heavily, SMOTE generates brand-new synthetic fraud examples by interpolating between real ones. This literally adds more fraud samples to the training set.

Important rule

SMOTE is applied only to the training data, never the test data. The test set must represent reality — if we generated fake fraud in the test set, our performance metrics would not reflect how the model performs in the real world.

Experiment 4 — Variance Threshold

Some features barely change across all transactions and therefore carry very little information. Variance Threshold removes these low-signal features automatically. The threshold is set to 0.01, meaning any feature with a variance below this is discarded.

Experiment 5 — SMOTE + Variance Threshold

A combination approach: SMOTE is applied first to balance the classes, then Variance Threshold removes low-information features. The hypothesis is that combining class balancing with feature cleaning will produce better results than either technique alone.

Experiment 6 — Balanced Subsample

A variant of class weighting specific to Random Forest. Instead of calculating class weights from the full dataset, `balanced_subsample` calculates weights separately for each bootstrap sample used to build each tree. This introduces more diversity into how imbalance is handled across the forest.

Experiment 7 — Custom Class Weight 1:5

Rather than letting the algorithm calculate weights automatically, we manually assign fraud transactions five times the weight of legitimate ones. This gives us direct control over how aggressively the model chases fraud cases.

Experiment 8 — Custom Class Weight 1:10

The weight ratio is pushed even further — fraud cases are now ten times more important than legitimate ones. This makes the model more aggressive at catching fraud, likely at the cost of more false alarms.

Experiment 9 — XGBoost

XGBoost (Extreme Gradient Boosting) is a fundamentally different algorithm from Random Forest. Where Random Forest builds many trees independently and averages them, XGBoost builds trees sequentially — each new tree learns from the mistakes of all previous trees.

The `scale_pos_weight` parameter is used to handle class imbalance, set to the ratio of negative to positive cases (legitimate to fraudulent transactions).

Why XGBoost often wins on tabular data

Sequential learning means later trees specifically focus on the cases that earlier trees got wrong. This iterative error-correction often produces significantly better results than averaging independent trees, especially on complex fraud patterns.

5. Results & Comparison

All experiments are evaluated using three metrics. Here is what each means in plain English:

Metric	Question it answers	Formula
Precision	Of all transactions flagged as fraud, how many actually were fraud?	$\text{True Positives} / (\text{True Positives} + \text{False Positives})$
Recall	Of all actual fraud cases, how many did the model catch?	$\text{True Positives} / (\text{True Positives} + \text{False Negatives})$
F1 Score	What is the overall balance between precision and recall?	$2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$

XGBoost achieved the best F1 Score across all experiments. The full comparison table (sorted by F1 Score) from the notebook demonstrates that sequential boosting significantly outperforms the Random Forest-based approaches for this fraud detection task.

Key takeaway: Handling class imbalance helps across all methods, but the choice of algorithm matters more than the imbalance strategy.

6. Threshold Optimisation

By default, a classifier predicts "fraud" when the fraud probability is above 0.5 (50%). But this default is rarely optimal. By adjusting this threshold, we can control the trade-off between precision and recall.

Threshold	Precision	Recall	F1 Score
0.1	Low	Very High	Moderate
0.3	Moderate	High	Moderate-High
0.5 (default)	Moderate-High	Moderate	Moderate
0.8 (selected)	90.9%	90.7%	90.8%

Why 0.8 was chosen

At threshold 0.8, only transactions where the model is very confident ($\geq 80\%$ sure) are flagged as fraud. This produces the best balance between precision and recall, achieving over 90% on both metrics. Raising the threshold further would increase precision but miss too many actual frauds.

The ROC-AUC score of approximately 0.999 confirms that the XGBoost model has near-perfect ability to distinguish between fraudulent and legitimate transactions at any threshold — the threshold then controls where we draw the line.

7. Feature Importance Analysis

After training XGBoost, the model reports how much each feature contributed to its decisions. This is called feature importance and helps us understand what the model learned.

Most Important Features

- `amt` — the transaction amount was the single strongest predictor of fraud
- Transaction category columns — where money was spent (grocery, entertainment, etc.) was highly informative
- Transaction timing — the hour and `unix_time` of the transaction were important
- `past_avg_amt` — the customer's historical average spend was a key reference point

Low-Importance Features (removed)

Feature importance analysis revealed that several engineered features contributed very little:

- `distance` — geographic distance between customer and merchant
- `merchant_count` — how many times the merchant had appeared before
- `amt_deviation` — the ratio of current amount to historical average
- `time_since_last_txn` — seconds since the customer's last transaction

What happened after removing them?

The model was retrained without these four features. Performance remained nearly identical: Precision 90.3%, Recall 91.1%, F1 90.7%, ROC-AUC ~0.999. This confirms the features were genuinely redundant. The simpler model is better — fewer features means faster inference, easier maintenance, and less risk of overfitting.

8. Final Model Performance

The final model is XGBoost trained on the reduced feature set (low-importance features removed), evaluated at a decision threshold of 0.8.

Metric	Score
Precision	90.3%
Recall	91.1%
F1 Score	90.7%
ROC-AUC	~0.999
Frauds correctly caught	406 out of 429
Legitimate transactions falsely flagged	~117

In plain English: out of 429 fraudulent transactions in the test set, the model correctly identified 406 of them. It raised a false alarm on 117 legitimate transactions. Given that the test set contains tens of thousands of transactions, this false-positive rate is very low.

Business interpretation

A bank using this model would block 94.6% of fraud attempts. For every 100 fraud alerts raised, 90 would be genuine frauds. Investigators would need to manually review only a small number of false alarms — a highly practical outcome for a real deployment.

9. Key Technical Decisions

Several deliberate design choices throughout the project are worth highlighting:

Chronological sorting before feature engineering

The dataset was sorted by `unix_time` before any customer-history features were created. This ensures no future information accidentally leaks into past-looking features — a common mistake in time-series machine learning problems.

`shift(1)` for historical features

The `past_avg_amt` feature uses `shift(1)` to exclude the current transaction from the customer's historical average. Without this, the model would use information it wouldn't have at the moment of making a real-time prediction.

SMOTE on training data only

Synthetic examples are generated only in the training set. The test set remains unchanged so that evaluation metrics reflect real-world class distribution, not an artificially balanced one.

Frequency encoding over one-hot for high-cardinality columns

City and job columns contain hundreds of unique values. One-hot encoding them would create hundreds of columns, making the dataset wide and sparse. Frequency encoding represents each value as a single number — how often it appears — keeping the dataset compact.

Threshold tuning as a post-training step

Rather than accepting the default 0.5 threshold, multiple thresholds were evaluated systematically. This is a standard industry practice that is often skipped in academic projects but is essential in production systems where business requirements determine the acceptable precision-recall trade-off.

10. Conclusion

This project demonstrates the full pipeline of a production-oriented fraud detection system, from raw transaction data to a deployable model.

The most important lessons from the experiments:

- Handling class imbalance is essential — a naive baseline will always favour the majority class
- Algorithm choice matters more than imbalance strategy — XGBoost outperformed all Random Forest variants
- Threshold tuning is not optional — the right threshold can dramatically improve real-world usability
- Feature importance feedback loops are valuable — retraining on fewer, better features simplified the model without losing performance
- Data leakage prevention is non-negotiable — every temporal feature was engineered to only use information available at prediction time

Final model

XGBoost with `scale_pos_weight` balancing, trained on the reduced feature set, evaluated at threshold 0.8. Achieves 90.3% Precision, 91.1% Recall, 90.7% F1, and ~0.999 ROC-AUC. Saved as `fraud_detection_xgboost.pkl`.

— End of Document —